🤖 TİRNAK TARAYICI SİSTEMİ - MODÜLER KOD YAPISI DOKÜMANTASYONu

📋 İÇİNDEKİLER

1. Dosya Yapısı
2. Modül Detayları
3. Çalışma Akışları
4. Veri Akışı
5. Konfigürasyon Noktaları
6. Kullanım Senaryoları

📁 DOSYA YAPISI

```
nail_scanner/
├── shared_config.py              # Ortak ayarlar merkezi
├── master_controller.py          # Ana koordinatör (Phase 1+2)
├── imaging_pipeline.py           # Görüntü işleme pipeline
├── motor_controller.py           # 3 eksenli motor kontrolü
├── motor_camera_calibrator.py    # Motor-kamera hizalama aracı
├── camera_calibration_*.json     # Kamera kalibrasyon verileri
└── scan_outputs/                 # Çıktı dosyaları
    ├── all_nails_path_*.txt      # Birleşik koordinatlar
    ├── nail_*_path_*.txt         # Bireysel tırnak koordinatları
    └── multi_nail_results_*.png  # Görselleştirme sonuçları
```

📖 MODÜL DETAYLARI

1. **shared_config.py** - Ortak Ayarlar Merkezi

Veri Yapıları:

python

```python
PATHS = {
    'MODEL_PATH': './0824.tflite',
    'CALIBRATION_FILE': 'camera_calibration_*.json',
    'OUTPUT_DIR': 'scan_outputs',
    'TEMP_DIR': 'temp_images'
}

CAMERA_CONFIG = {
    'IMAGE_SIZE': (1920, 1080),
    'CAMERA_CONTROLS': {
        "AfMode": "Continuous",
        "ExposureTime": 66000,
        "Brightness": 0.1,
        # ... diğer kamera parametreleri
    }
}

PATH_CONFIG = {
    'POINT_SPACING': 0.5,          # mm - nokta aralığı
    'STRIP_WIDTH': 1.0,            # mm - şerit genişliği
    'MIN_NAIL_AREA_MM2': 50,       # Minimum tırnak alanı
    'MAX_NAIL_AREA_MM2': 500,      # Maksimum tırnak alanı
}

MOTOR_CONFIG = {
    'X_STEP_PIN': 17, 'X_DIR_PIN': 27,
    'Y_STEP_PIN': 22, 'Y_DIR_PIN': 23,
    'Z_STEP_PIN': 24, 'Z_DIR_PIN': 25,
    'ENABLE_PIN': 18, 'SWITCH_PIN': 16,
    'SPEED': 100,                  # X,Y hızı (adım/saniye)
    'Z_AXIS_SPEED': 50,           # Z hızı (adım/saniye)
    'MAX_TRAVEL': 100,            # Maksimum hareket sınırı (mm)
}

AI_CONFIG = {
    'PATCH_SIZE': (512, 512),
    'STRIDE': (256, 256),
    'CONFIDENCE_THRESHOLD': 0.5,
}

SYSTEM_CONFIG = {
    'ENABLE_AI_MODEL': True,       # False = Mock segmentation
    'ENABLE_VISUALIZATION': True,  # Grafik gösterimi
    'SAVE_DEBUG_IMAGES': True,    # Debug fotoğrafları
}
```

Ana Fonksiyonlar:

- `get_motor_calculations()` - Adım/mm hesaplamaları
- `create_directories()` - Gerekli klasörleri oluşturma
- `validate_config()` - Dosya varlık kontrolleri
- `print_system_info()` - Sistem bilgilerini yazdırma

---

2. **master_controller.py** - Ana Koordinatör

NailScannerMaster Sınıfı:

Temel Özellikler:

- Phase 1 (Imaging) ve Phase 2 (Motor) koordinasyonu
- Subprocess ile bellek izolasyonu
- Otomatik dosya temizleme
- Sistem gereksinim kontrolleri

Ana Metodlar:

```python
check_system_requirements()      # Modül ve dosya kontrolleri
run_imaging_pipeline()           # Phase 1: Subprocess olarak çalıştırma
run_motor_controller()           # Phase 2: Import ederek çalıştırma
run_full_pipeline()              # Phase 1 + 2 sıralı çalıştırma
show_coordinates_info()          # Koordinat dosyası analizi
cleanup_related_files()         # Başarılı tarama sonrası dosya silme
```

Menü Seçenekleri:

1. Tam Otomatik Tarama (Phase 1 + 2)
2. Sadece Görüntü İşleme (Phase 1)
3. Sadece Motor Tarama (Phase 2)
4. Sistem Durumu
5. Konfigürasyon Bilgileri
6. Çıkış

---

3. **imaging_pipeline.py** - Görüntü İşleme Pipeline

NailImagingPipeline Sınıfı:

Pipeline Adımları:

1. **Kamera Kurulumu** → shared_config'den ayarları al

2. **Canlı Fotoğraf Çekme** → Q tuşu ile yakala
3. **AI/Mock Segmentasyon** → Tırnak maskesi oluştur
4. **Kontur Filtreleme** → Boyut kontrolü (50-500 mm²)
5. **Path Planning** → Her tırnak için zigzag raster scan
6. **Koordinat Dönüşümü** → Piksel → gerçek mm koordinatları
7. **Dosya Kaydetme** → Bireysel + birleşik koordinat dosyaları
8. **Görselleştirme** → 2x2 subplot sonuç grafiği

Ana Metodlar:

```python
load_calibration()               # Kamera kalibrasyon verilerini yükle
setup_camera()                   # Picamera2 başlatma ve ayarlama
capture_image_interactive()      # Canlı önizleme + fotoğraf çekme
run_ai_segmentation()            # TensorFlow Lite patch-wise işleme
create_mock_segmentation()       # Test amaçlı eliptik tırnak
filter_nail_contours()           # Boyut ve alan filtreleme
create_raster_scan_path()        # Zigzag tarama path'i oluşturma
convert_to_motor_coordinates()   # Kalibrasyon matrisi ile dönüşüm
generate_paths_for_all_nails()   # Çoklu tırnak koordinat üretimi
save_coordinates()               # TXT formatında dosya kaydetme
visualize_results()              # Matplotlib ile sonuç gösterimi
```

Çıktı Dosyaları:

- `all_nails_path_YYYYMMDD_HHMMSS.txt` - Ana koordinat dosyası
- `nail_N_path_YYYYMMDD_HHMMSS.txt` - Bireysel tırnak dosyaları
- `multi_nail_results_YYYYMMDD_HHMMSS.png` - Görselleştirme

---

4. **motor_controller.py** - Motor Kontrol Sistemi

Veri Yapıları:

```python
@dataclass
class Position:
    x: float = 0.0    # X pozisyonu (mm)
    y: float = 0.0    # Y pozisyonu (mm)
    z: float = 0.0    # Z pozisyonu (mm)

@dataclass
class ScanPoint:
    x: float          # X koordinatı (mm)
    y: float          # Y koordinatı (mm)
    nail_id: int = 0  # Tırnak ID'si
    index: int = 0    # Nokta sıra numarası
```

NailScannerController Sınıfı:

Ana Metodlar:

```python
setup_gpio()                    # 3 eksen + switch pin konfigürasyonu
load_coordinates_from_file()    # TXT dosyasından koordinat okuma
mm_to_steps()                   # mm → motor adım dönüşümü
move_motor()                    # Tekli motor hareket kontrolü
move_to_position()              # X,Y pozisyonlama
perform_z_axis_scan()           # Z ekseni switch'e kadar inme
retract_z_axis()                # Z ekseni güvenli geri çekme
run_full_scan()                 # Tüm koordinatları sıralı tarama
home_all_axes()                 # Ana pozisyona dönme (0,0,0)
cleanup()                       # GPIO temizleme ve güvenli kapatma
```

Tarama Süreci:

1. **Koordinat Yükleme** → TXT dosyasından parse etme
2. **Pozisyon Hareket** → X,Y hedef koordinatına gitme
3. **Z Ekseni Tarama** → Switch tetiklenene kadar inme
4. **Veri Toplama** → 0.1s bekleme (simülasyon)
5. **Güvenli Geri Çekme** → Z eksenini 5mm yukarı çekme
6. **Sonraki Nokta** → Sıradaki koordinata geçme

Güvenlik Özellikleri:

- Z ekseni limit switch koruması
- Hareket sınır kontrolleri (±100mm)
- Acil durdurma (Ctrl+C)

- Motor sürücü durumu kontrolü
- GPIO temizleme garantisi

---

5. **motor_camera_calibrator.py - Kalibrasyon Aracı**

MotorCameraCalibrator Sınıfı:

Amaç: Motor sıfır noktası ile kamera origin'ini hizalamak

Ana Metodlar:

```python
load_calibration()              # Kamera kalibrasyon matrisi yükleme
setup_gpio()                    # 3 eksen motor pin konfigürasyonu
setup_camera()                  # Aynı kamera ayarlarıyla başlatma
check_z_switch()                # Z limit switch durumu kontrol
move_motor_steps()              # Adım bazında motor hareketi
draw_calibration_overlay()      # Kamera üzerine grid + origin çizme
run_calibration()               # Ana kalibrasyon döngüsü
go_home()                       # 3 eksen home pozisyonu
```

Görsel Özellikler:

- **Kırmızı çapraz:** Kamera origin noktası
- **Grid çizgileri:** 50 piksel aralıklı referans
- **Pozisyon bilgisi:** X,Y,Z motor koordinatları
- **Switch durumu:** Z limit switch real-time gösterimi
- **Kontrol rehberi:** Tuş fonksiyonları açıklaması

Tuş Kontrolleri:

- **W/S:** Y ekseni hareket (+/-)
- **A/D:** X ekseni hareket (+/-)
- **R/F:** Z ekseni hareket (yukarı/aşağı)
- **+/-:** Adım boyutu ayarlama (1-100)
- **H:** Home pozisyonuna gitme (0,0,0)
- **T:** Pozisyon sayacını sıfırlama
- **Q:** Çıkış

🔄 ÇALIŞMA AKIŞLARI

Senaryo 1: Tam Otomatik Tarama

bash

```
python3 master_controller.py
# Menüden "1" seç
```

Akış:

```
Master Controller
    ↓
Subprocess: imaging_pipeline.py --save-coordinates
    ↓ (Bellek izolasyonu)
Koordinat dosyaları oluştur (all_nails_path_*.txt)
    ↓
Import: motor_controller.py
    ↓
Koordinatları yükle ve tara
    ↓
Başarılı tarama sonrası dosyaları sil
```

Senaryo 2: Manuel İki Aşama

bash

```
# Phase 1
python3 imaging_pipeline.py --save-coordinates

# Phase 2
python3 motor_controller.py -f all_nails_path_20250522_180000.txt --auto-scan
```

Senaryo 3: Kalibrasyon

bash

```
python3 motor_camera_calibrator.py
# W,A,S,D,R,F tuşlarıyla manuel hizalama
# Motor origin → kamera origin hizalama
```

📊 VERİ AKIŞI

```
📷 Kamera Görüntüsü (1920x1080)
    ↓
🤖 AI Segmentasyon (512x512 patch'ler)
    ↓
🔍 Kontur Filtreleme (50-500 mm² boyut)
    ↓
🗺️ Path Planning (zigzag raster scan)
    ↓
📐 Koordinat Dönüşümü (piksel → mm)
    ↓
💾 Dosya Kaydetme (TXT + PNG)
    ↓
🤖 Motor Yükleme (TXT parse)
    ↓
🎯 3D Tarama (X,Y,Z hareket)
    ↓
🧹 Otomatik Temizlik (dosya silme)
```

🔧 ÖNEMLİ KONFIGÜRASYON NOKTALARI

Motor Ayarları (shared_config.py):

python

```python
MOTOR_CONFIG = {
    # Pin Atamaları
    'X_STEP_PIN': 17, 'X_DIR_PIN': 27,    # X ekseni
    'Y_STEP_PIN': 22, 'Y_DIR_PIN': 23,    # Y ekseni
    'Z_STEP_PIN': 24, 'Z_DIR_PIN': 25,    # Z ekseni
    'ENABLE_PIN': 18,                      # Motor sürücü enable
    'SWITCH_PIN': 16,                      # Z limit switch

    # Hareket Parametreleri
    'SPEED': 100,                          # X,Y hızı (adım/s)
    'Z_AXIS_SPEED': 50,                    # Z hızı (adım/s)
    'MAX_TRAVEL': 100,                     # Hareket sınırı (mm)
    'PAUSE_BETWEEN_POINTS': 0.5,           # Nokta arası bekleme (s)
    'Z_RETRACT_DISTANCE': 5.0,             # Z geri çekme (mm)

    # Mekanik Parametreler
    'MICROSTEP': 8,                        # 1/8 mikro adım
    'STEPS_PER_REV': 200,                  # Motor adım/tur
    'PULLEY_TEETH': 20,                    # Kasnak diş sayısı
    'GT2_PITCH': 2,                        # GT2 kayış pitch (mm)
}
```

Path Planning Ayarları:

```python
PATH_CONFIG = {
    'POINT_SPACING': 0.5,      # Tarama noktaları arası mesafe (mm)
    'STRIP_WIDTH': 1.0,        # Tarama şerit genişliği (mm)
    'MIN_NAIL_AREA_MM2': 50,   # Minimum tırnak alanı (mm²)
    'MAX_NAIL_AREA_MM2': 500,  # Maksimum tırnak alanı (mm²)
}
```

Kamera Ayarları:

```python
CAMERA_CONFIG = {
    'IMAGE_SIZE': (1920, 1080),          # Full HD çözünürlük
    'CAMERA_CONTROLS': {
        "ExposureTime": 66000,            # Pozlama süresi
        "AnalogueGain": 4.0,              # Analog kazanç
        "Brightness": 0.1,                # Parlaklık
        "Contrast": 1.3,                  # Kontrast
        "Sharpness": 3.0,                 # Netlik
    }
}
```

Sistem Davranış Ayarları:

```python
SYSTEM_CONFIG = {
    'ENABLE_AI_MODEL': True,       # False: Mock segmentation kullan
    'ENABLE_VISUALIZATION': True,  # Grafik gösterimi açık/kapalı
    'SAVE_DEBUG_IMAGES': True,    # Debug fotoğrafları kaydet
}
```

🎯 KULLANIM SENARYOLARI

1. Geliştirme ve Test:

- AI modeli olmadan test: `ENABLE_AI_MODEL = False`
- Görselleştirme kapalı: `ENABLE_VISUALIZATION = False`
- Hızlı test için: `POINT_SPACING = 2.0` (daha seyrek)

2. Prodüksiyon Kullanımı:

- Tam otomatik mod: `master_controller.py`
- Dosya otomatik temizlik: Başarılı tarama sonrası silme
- Hata durumu koruması: Başarısız taramada dosya korunur

3. Kalibrasyon ve Kurulum:

- İlk kurulum: `motor_camera_calibrator.py`
- Motor-kamera hizalama: Manuel tuş kontrolü
- Kamera kalibrasyon: Ayrı chessboard kalibrasyonu

4. Sorun Giderme:

- Modül kontrolleri: `master_controller.py` → "4. Sistem Durumu"
- Konfigürasyon testi: `python3 shared_config.py`
- Manuel motor testi: `motor_controller.py` menü modu

📝 NOTLAR

Bellek Yönetimi:

- Imaging pipeline subprocess olarak çalışır (izolasyon)
- Her phase sonrası `gc.collect()` çağrılır
- Başarısız işlemler sonrası dosya temizliği

Güvenlik Özellikleri:

- GPIO temizleme garantisi (`finally` blokları)
- Motor hareket sınır kontrolleri
- Z ekseni limit switch koruması
- Acil durdurma destegi (Ctrl+C)

Dosya Yönetimi:

- Timestamp bazlı dosya isimlendirme
- Başarılı tarama sonrası otomatik temizlik
- Hata durumunda dosya korunması
- İlgili dosyaların grup halinde silinmesi

Son Güncelleme: 2025-06-02
Versiyon: 1.0
Geliştirici: Tırnak Tarayıcı Sistemi